

PANDAS LIBRARY USE CASES IN DATA SCIENCE

AIM

To understand the use cases of DataFrames using the pandas library.

PROGRAM

```
import numpy as np
import pandas as pd

# Creating a DataFrame from a list
data_list = [[1, 'Smith', 50000], [2, 'Jones', 60000]]
df = pd.DataFrame(data_list)
print(df)
df.columns = ['Empd', 'Name', 'Salary']
print(df)
df.info()

# Reading a CSV file
df = pd.read_csv("50_Startups.csv")
df.info()
print(df.head())
print(df.tail())

# Employee data
df = pd.read_csv("employee.csv")
print(df.head())
print(df.tail())
df.info()

print(df.salary)
print(type(df.salary))
print("Mean      :", df.salary.mean())
print("Median    :", df.salary.median())
print("Mode      :\n", df.salary.mode())
print("Variance  :", df.salary.var())
print("Std dev   :", df.salary.std())

print(df.describe())
print(df.describe(include='all'))

empCol = df.columns
print(empCol)
emparray = df.values
print(emparray)
employee_DF = pd.DataFrame(emparray, columns=empCol)
print(employee_DF)
```

OUTPUT

```
   0    1    2
0  1  Smith  50000
1  2  Jones  60000
Empd  Name  Salary
0     1  Smith   50000
1     2  Jones   60000
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Empd      2 non-null      int64
```

```

1  Name      2 non-null      str
2  Salary    2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 180.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   R&D Spend              10 non-null    float64
1   Administration         10 non-null    float64
2   Marketing Spend        10 non-null    float64
3   State                  10 non-null    str
4   Profit                 10 non-null    float64
dtypes: float64(4), str(1)
memory usage: 532.0 bytes
   R&D Spend  Administration  Marketing Spend   State   Profit
0  165349.20    136897.80      471784.10   New York  192261.83
1  162597.70    151377.59      443898.53  California  191792.06
2  153441.51    101145.55      407934.54   Florida  191050.39
3  144372.41    118671.85      383199.62   New York  182901.99
4  142107.34     91391.77      366168.42   Florida  166187.94
   R&D Spend  Administration  Marketing Spend   State   Profit
5    1000.23    124153.04        1903.93   New York  64926.08
6    1315.46    115816.21     297114.46   Florida  49490.75
7         0.00    135426.92         0.00  California  42559.73
8     542.05     51743.15         0.00   New York  35673.41
9         0.00    116983.80     45173.06  California  14681.40

emp id      name      salary
0         1  SREE VARSSINI K S    5000
1         2    SREEMATHI B      6000
2         3    SREYA G          7000
3         4  SREYASKARI MULLAPUDI    5000
4         5    SRI AKASH U G      8000

emp id      name      salary
2         3    SREYA G          7000
3         4  SREYASKARI MULLAPUDI    5000
4         5    SRI AKASH U G      8000
5         6  SRI HARSHAVARDHANAN R    3000
6         7  SRI HARSHAVARDHANAN R    6000
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   emp id  7 non-null      int64
1   name    7 non-null      str
2   salary  7 non-null      int64
dtypes: int64(2), str(1)
memory usage: 300.0 bytes
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64
<class 'pandas.Series'>
Mean      : 5714.285714285715
Median    : 6000.0
Mode      :
0         5000
1         6000
Name: salary, dtype: int64
Variance : 2571428.5714285714
Std dev  : 1603.5674514745463
emp id      salary
count  7.000000    7.000000
mean    4.000000  5714.285714
std     2.160247  1603.567451
min     1.000000  3000.000000
25%     2.500000  5000.000000

```

```

50%    4.000000    6000.000000
75%    5.500000    6500.000000
max     7.000000    8000.000000

   emp id      name  salary
count  7.000000      7    7.000000
unique   NaN      6      NaN
top      NaN  SRI HARSHAVARDHANAN R    NaN
freq     NaN      2      NaN
mean    4.000000      NaN  5714.285714
std     2.160247      NaN  1603.567451
min     1.000000      NaN  3000.000000
25%     2.500000      NaN  5000.000000
50%     4.000000      NaN  6000.000000
75%     5.500000      NaN  6500.000000
max     7.000000      NaN  8000.000000
Index(['emp id', 'name ', 'salary'], dtype='str')
[[1 'SREE VARSSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
 [5 'SRI AKASH U G' 8000]
 [6 'SRI HARSHAVARDHANAN R' 3000]
 [7 'SRI HARSHAVARDHANAN R' 6000]]
   emp id      name  salary
0      1  SREE VARSSINI K S  5000
1      2   SREEMATHI B    6000
2      3    SREYA G    7000
3      4  SREYASKARI MULLAPUDI  5000
4      5    SRI AKASH U G    8000
5      6  SRI HARSHAVARDHANAN R  3000
6      7  SRI HARSHAVARDHANAN R  6000

```

RESULT

Thus the DataFrame creation, loading, inspection and statistical summary operations were performed using the pandas library.